ML Experiment-03

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
confusion_matrix, accuracy_score, precision_score, recall_score,
classification_report
)
```

In [2]:
```
df = pd.read_csv("Social_Network_Ads.csv")
print("Dataset Preview:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Preview:
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   User ID          400 non-null    int64
 1   Gender           400 non-null    object
 2   Age              400 non-null    int64
 3   EstimatedSalary  400 non-null    int64
 4   Purchased        400 non-null    int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
None

Missing Values:
User ID            0
Gender             0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

In [3]:
```
df = df.drop_duplicates().reset_index(drop=True)
```

In [5]:
```
if "Gender" in df.columns:
    df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
```

In [6]:
```
X = df.drop(columns=["Purchased", "User ID"], errors="ignore")
y = df["Purchased"]

X = X.select_dtypes(include=[np.number])
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.25, random_state=42, stratify=y
)
```

In [9]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
k=5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)
```

In [10]:
```
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
print("\nConfusion Matrix:")
print(cm)
print("\nAccuracy :", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall   :", recall)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Confusion Matrix:
[[59  5]
 [ 4 32]]

Accuracy : 0.91
Error Rate: 0.08999999999999997
Precision: 0.8648648648648649
Recall   : 0.8888888888888888

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.92      0.93        64
           1       0.86      0.89      0.88        36

    accuracy                           0.91       100
   macro avg       0.90      0.91      0.90       100
weighted avg       0.91      0.91      0.91       100
```

In [11]:
```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("KNN Confusion Matrix")
plt.show()
```

